.

Universidad Católica Boliviana “San Pablo”

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

Postgrado

ESPECIALIDAD EN: “ARQUITECTURA Y DESARROLLO AVANZADO DE SOFTWARE”

PROYECTO DE POSTGRADO

“Desarrollo de aplicación móvil Full Stack orientado a la gestión de microaprendizaje docente en educación secundaria basada en arquitectura limpia”

Realizado por:

Pablo Benjamin Huañapaco Peña

Revisor: Msc. Orlando Rivera Jurado

La Paz - Bolivia

Febrero 2025

2.- Marco Referencial

2.1.- Introducción

En el panorama educativo actual, los docentes enfrentan el desafío de integrar innovaciones digitales y pedagógicas de manera constante. Mientras que hace años bastaba dominar las metodologías tradicionales de enseñanza, hoy es indispensable que los maestros de educación regular integren constantemente nuevas herramientas digitales, enfoques pedagógicos disruptivos y estrategias para conectar con los estudiantes. Sin embargo, los programas habituales de capacitación docente presentan serias deficiencias: horarios rígidos, contenidos genéricos y formatos extensos que no se centran en la información de manera práctica. Esta situación genera que la actualización profesional de los docentes sea muy complicada bajo los esquemas convencionales.

Aquí es donde surge el potencial del microaprendizaje, una modalidad que ofrece clases formativas de 5 a 15 minutos adaptadas al perfil de cada docente. Para materializar esta solución en el contexto de Bolivia, este proyecto propone el desarrollo de una plataforma móvil bajo un enfoque Full Stack, utilizando Flutter y Firebase y fundamentada en Arquitectura Limpia. La relevancia técnica y organizacional de este sistema radica en su capacidad de ofrecer una herramienta escalable, segura y de alta usabilidad que democratice el acceso a formación continua de calidad.

El presente documento se estructura iniciando con el marco referencial, los objetivos y la justificación técnica del proyecto; posteriormente, se detalla el marco tecnológico, el análisis de requerimientos y el diseño arquitectónico del sistema; para finalmente describir el desarrollo Full Stack, la estrategia de pruebas, el despliegue de infraestructura y la evaluación técnica de los resultados y conclusiones alcanzadas.

2.2.- Antecedentes

En el contexto de la digitalización educativa y el desarrollo de competencias docentes, se puede evidenciar la viabilidad y el impacto positivo del microaprendizaje en la capacitación de profesores. Se demostró una mejora significativa en la adquisición de competencias digitales a través de módulos formativos cortos y accesibles, utilizando una arquitectura web robusta (García y López, 2022).

Asimismo, la integración de la Inteligencia Artificial para la personalización de rutas de aprendizaje en la formación docente ha sido explorada con un enfoque Full Stack, se utilizó algoritmos de aprendizaje automático para adaptar el contenido y la dificultad de los módulos de capacitación según el progreso y las necesidades individuales de los docentes, resultando en una mayor motivación y eficiencia en el aprendizaje (Rodríguez y Pérez, 2023).

En cuanto a la aplicación de tecnologías Full Stack para el desarrollo de plataformas educativas, se presenta un caso exitoso de implementación que

subraya la versatilidad y el rendimiento que estas arquitecturas ofrecen para manejar grandes volúmenes de usuarios y contenidos educativos dinámicos, sentando las bases para sistemas escalables de formación continua (Martínez, 2021).

Se puede destacar cómo las certificaciones de microformaciones están ganando terreno como una forma flexible y reconocida de acreditar competencias docentes, facilitando su inserción y progresión en un mercado laboral en constante cambio (OECD Education and Skills, 2022).

Se observa que los Sistemas de Gestión de Aprendizaje (LMS) tradicionales como Moodle o Blackboard fueron concebidos bajo un paradigma de escritorio y aprendizaje extensivo, y no así para consumo granular, lo que deriva en interfaces saturadas. Hoy en día lo que se busca es soluciones de diseño bajo la premisa de 'Mobile-First'. Mientras que un LMS convencional requiere sesiones prolongadas y flujos de navegación complejos

2.3 .- Descripción del objeto de estudio

El objeto de estudio de este proyecto es el proceso de gestión y distribución de microaprendizaje mediante mediante el desarrollo de una aplicación móvil Full Stack, orientado específicamente a la actualización pedagógica de docentes de educación secundaria. Este estudio se centra en el diseño e implementación de una infraestructura tecnológica desacoplada que supere las limitaciones de los sistemas de gestión de aprendizaje (LMS) tradicionales, como la rigidez operativa y la necesidad de conexión para el acceso al contenido. Se analiza cómo la integración de una arquitectura limpia (Clean Architecture) y el uso de servicios en la nube con capacidades de persistencia local facilitan una entrega de contenido granular; esto permite al usuario acceder a videos de las clases, cuestionarios de conocimientos y material de lectura descargable directamente en el dispositivo, logrando una solución eficiente y adaptable a la realidad tecnológica del magisterio boliviano.

2.4.- Identificación y formulación del problema de investigación

El sistema de capacitación y actualización para maestros de educación regular en Bolivia tiene varios desafíos que debe superar. Los programas actuales son predominantemente largos, genéricos y obligatorios, diseñados más para el cumplimiento administrativo que para atender las necesidades reales y urgentes del aula. Esta desconexión genera desmotivación y resistencia, socavando el desarrollo profesional de los maestros.

El problema radica en la falta de contextualización y personalización. Se ofrecen cursos que ignoran los complejos entornos en los que operan los maestros bolivianos. Un profesor de física de secundaria en El Alto tiene necesidades formativas radicalmente distintas a las de una maestra de matemática en una comunidad rural de Potosí. Por ejemplo, se proponen módulos sobre robótica o programación sin considerar que muchas escuelas rurales carecen de acceso a internet estable y equipamiento tecnológico básico. Esto no solo vuelve la

capacitación irrelevante, sino que también frustra a los docentes al proponer soluciones inviables.

Otro aspecto a tomar en cuenta es la falta de motivación y resistencia que tiene los docentes para actualizarse especialmente en el área urbano. Tomando en cuenta los datos estadísticos de maestros que se inscribieron en cursos de formación continua en las diferentes instituciones estatales que imparten cursos enfocados al magisterio de Bolivia, se puede notar que la mayoría son de áreas dispersas y en su mayoría no por interés sino por obligación de las autoridades distritales.

2.5 . Planteamiento y Formulación del Problema

2.5.1.- Situación actual

Actualmente, el proceso de actualización docente en el magisterio de Bolivia se gestiona a través de modelos de formación continua centralizados y tradicionales. Este sistema se caracteriza por ciclos formativos de larga duración (40 a 60 horas) con contenidos genéricos que no discriminan entre contextos urbanos, rurales o especialidades técnicas. A nivel tecnológico, estos procesos dependen de plataformas de gestión de aprendizaje (LMS) convencionales como Moodle o Canvas, las cuales presentan arquitecturas pesadas y monolíticas diseñadas originalmente para entornos de escritorio y conectividad de banda ancha, lo que resulta ineficiente para el docente que intenta capacitarse desde dispositivos móviles de gama media o en zonas con señal intermitente.

A nivel operativo, existe una brecha crítica entre la oferta formativa y la infraestructura tecnológica de las unidades educativas. Mientras se proponen contenidos de vanguardia como robótica o programación, la realidad técnica en campo muestra carencias en conectividad estable y equipamiento básico. Estadísticamente, la participación docente en estos programas, especialmente en el área urbana, es baja y reactiva; los maestros se inscriben mayoritariamente por cumplimiento administrativo o presión de las autoridades distritales, más que por una motivación de desarrollo profesional. El sistema actual carece de mecanismos de personalización, lo que resulta en una subutilización de recursos estatales y una resistencia creciente por parte del magisterio.

2.5.2.- Problema central

El problema central radica en la ineficiencia técnica y arquitectónica de los sistemas de gestión de aprendizaje (LMS) actuales, los cuales operan bajo estructuras monolíticas que no logran adaptarse a las limitaciones de infraestructura del magisterio boliviano. Desde una perspectiva de desarrollo Full Stack, esto se traduce en una alta dependencia de conectividad constante, un consumo excesivo de recursos en el cliente y la falta de una capa de servicios optimizada para la movilidad. Esta rigidez tecnológica impide la entrega de contenidos granulares (microaprendizaje) de forma fluida, forzando al docente a interactuar con plataformas lentas y complejas que no ofreceen persistencia de datos offline ni una sincronización eficiente entre el backend y los dispositivos de gama media.

2.5.3.- Planteamiento de la pregunta de Formulación del Problema

¿Cómo una aplicación móvil Full stack de gestión de microaprendizaje puede optimizar la gestión del microaprendizaje para docentes de secundaria en Bolivia, superando las limitaciones de rigidez, dependencia de conectividad y alto consumo de recursos de las plataformas tradicionales, facilitando así una actualización pedagógica ágil y personalizada?

3.- Objetivos

3.1.- Objetivo General

Desarrollar una aplicación móvil Full Stack basada en una arquitectura limpia (Clean Architecture) y prácticas de seguridad avanzada alineada a recomendaciones OWASP, que permita a los docentes de secundaria en Bolivia acceder a un modelo de gestión de microaprendizaje personalizado, facilitando la actualización de competencias pedagógicas mediante el consumo de contenidos granulares.

3.2.- Objetivos Específicos

- Diseñar la arquitectura de software basada en los principios de Clean Architecture, definiendo la separación de capas (Data, Domain, Presentation) para garantizar la independencia de marcos de trabajo y facilitar el mantenimiento evolutivo del sistema.
- Implementar la interfaz de usuario mediante el framework Flutter, aplicando patrones de diseño responsivo para asegurar una experiencia de usuario (UX) intuitiva que reduzca la curva de aprendizaje tecnológica de los docentes.
- Desarrollar el backend y la persistencia de datos mediante los servicios de Firebase, para permitir la distribución ágil de contenidos de microaprendizaje y la sincronización en tiempo real entre dispositivos.
- Configurar protocolos de seguridad mediante Firebase Authentication y Cloud Firestore Security Rules, para salvaguardar la privacidad de la información personal y los registros de avance pedagógico de los usuarios.
- Validar la estabilidad técnica y la aceptación pedagógica mediante pruebas unitarias y pruebas de usabilidad con un grupo de control, para asegurar que la herramienta cumple con los estándares de calidad y las expectativas de los maestros bolivianos.
- Implementar mecanismos de seguridad avanzada siguiendo las recomendaciones OWASP, con el propósito de proteger la información sensible de los usuarios.
- Desplegar la infraestructura necesaria y generar el paquete instalable (APK), para poner la aplicación a disposición de los usuarios finales en el entorno de producción.

3.3.- Validación de objetivos (SMART)

S - Specific (Específico)

Cumple.
¿Qué se va a desarrollar?
Una aplicación móvil Full Stack. No se trata de un prototipo limitado ni de una web adaptada, sino de un sistema completo que abarca desde la interfaz de usuario (Frontend) hasta la lógica de servidor y base de datos (Backend).

¿Quiénes son los beneficiarios?
Los docentes de educación secundaria en Bolivia. El software está diseñado específicamente para este perfil profesional, considerando sus necesidades de actualización pedagógica y su realidad tecnológica.

¿Cómo se va a construir? (Base Técnica)
Mediante la implementación de Clean Architecture (Arquitectura Limpia). Esto garantiza un sistema desacoplado donde la lógica de negocio es independiente de agentes externos como la base de datos o el framework de UI. Asimismo, se aplican estándares de seguridad OWASP para blindar la plataforma contra vulnerabilidades críticas.

¿Dónde se aplicará el impacto?
En el entorno de la formación continua y actualización de competencias. El sistema opera en un ecosistema móvil, permitiendo su uso en diversos contextos geográficos de Bolivia gracias a su enfoque en la movilidad.

¿Por qué es necesario? (Propósito)
Para superar la rigidez de los LMS tradicionales y permitir un modelo de gestión de microaprendizaje personalizado. El objetivo es transformar la capacitación docente de un formato extenso y genérico a uno granular, ágil y adaptado a la disponibilidad de tiempo del maestro.

M - Measurable (Medible)

Cumple. La medición de este objetivo se puede realizar a través de tres indicadores técnicos verificables en la segunda parte de tu proyecto:

- **Arquitectura:** Verificación de la separación de capas (Data, Domain, Presentation).
- **Seguridad:** Cumplimiento de la lista de verificación (checklist) de OWASP Mobile.
- **Funcionalidad:** Disponibilidad de los contenidos granulares en la interfaz móvil.

A - Achievable (Alcanzable)

Cumple. Es un proyecto ambicioso pero factible Se cuenta con el stack tecnológico adecuado, Flutter (Frontend) y Firebase (Backend) permite manejar la complejidad de la seguridad y la arquitectura dentro de los

plazos académicos. El alcance está delimitado a las funcionalidades críticas de microaprendizaje, evitando un "feature creep" (exceso de funciones) que ponga en riesgo la entrega.

R - Relevant (Relevante)

Cumple. El proyecto es vital porque propone una solución de ingeniería a un problema que los LMS tradicionales (Moodle, Blackboard) no han resuelto en el contexto boliviano: la falta de usabilidad móvil, la falta de personalización y la dependencia de internet estable. La inclusión de seguridad avanzada es crítica para proteger los registros pedagógicos de los docentes, elevando el valor profesional de la propuesta.

T - Time-bound (Temporal)

Cumple. El tiempo estimado para la finalización del proyecto es de 4 meses.

4.- Justificación

- Técnica (Arquitectura, escalabilidad, seguridad, automatización)

La elección del stack tecnológico responde a la necesidad de crear una herramienta de alto rendimiento con recursos optimizados. El uso de Flutter permite un desarrollo multiplataforma con un único código base, reduciendo tiempos de mantenimiento. Por otro lado, la implementación de Arquitectura Limpia (Clean Architecture) justifica la viabilidad técnica a largo plazo, ya que desacopla la lógica de negocio de las herramientas externas; esto permite que, aunque actualmente se utilice la capa gratuita de Firebase para la persistencia de datos y autenticación, el sistema pueda migrar a un servidor propio o una base de datos relacional en el futuro sin necesidad de reescribir la aplicación. Además, se justifica el uso de Firebase por su capacidad de ofrecer sincronización en tiempo real y notificaciones push, elementos críticos para la retención del usuario en modelos de microaprendizaje.

- Institucional / empresarial

Desde una perspectiva organizacional, el proyecto justifica su existencia al proponer una solución a la ineficiencia de los modelos de capacitación actuales en Bolivia. Al implementar una plataforma de microaprendizaje, las instituciones educativas pueden reducir los costos logísticos de los cursos presenciales y mejorar los indicadores de actualización docente. La herramienta actúa como un puente entre el contenido académico y la disponibilidad de tiempo del maestro, transformando la capacitación de una carga administrativa en un beneficio profesional accesible desde cualquier dispositivo móvil. Mientras que los LMS tradicionales suelen presentar barreras de usabilidad en dispositivos móviles y dependencia de una conectividad constante, la propuesta utiliza una arquitectura 'Mobile-First' con capacidades offline, facilitando que el maestro en entornos rurales o con conexión inestable pueda descargar contenidos. Así, la solución trasciende el enfoque administrativo y genérico de las plataformas actuales para

convertirse en una herramienta de actualización pedagógica ágil, personalizada y técnicamente resiliente ante la brecha digital boliviana.

- Académica (Aporte a la formación profesional del Full Stack Development)

Este proyecto representa un aporte significativo a la formación profesional en el área de Desarrollo Full Stack (FSD), al integrar de manera práctica la gestión de bases de datos NoSQL, el desarrollo de interfaces reactivas y la implementación de patrones de diseño avanzados. Académicamente, demuestra la capacidad de aplicar metodologías ágiles y principios de ingeniería de software para resolver problemas sociales complejos, validando las competencias adquiridas en el programa de especialidad en un entorno real y con usuarios finales específicos (maestros de secundaria).

- Metodológica (aporte a la metodología establecida en el proyecto)

La metodología adoptada justifica el rigor del proyecto al combinar el desarrollo iterativo con pruebas de usabilidad técnica y pedagógica. El uso del enfoque de microaprendizaje como base metodológica de enseñanza garantiza que el software sea coherente con su propósito formativo. Asimismo, la metodología de desarrollo asegura que cada fase del proyecto, desde el levantamiento de requerimientos mediante historias de usuario hasta el despliegue final, esté documentada y validada bajo estándares internacionales de calidad de software.

5.- Alcance y Delimitación

5.1.- Alcance

Se abarcará los siguientes módulos:

- **Módulo de Usuarios:** Gestión de perfiles docentes donde se almacenarán datos de especialidad, área de trabajo y niveles de progresión. Incluye la persistencia de datos personales en Cloud Firestore vinculados al UID único del usuario.
- **Módulo de Seguridad:** Implementación de un sistema de autenticación robusto mediante Firebase Authentication. Se desarrollarán flujos de registro, inicio de sesión y recuperación de credenciales, además de la configuración de *Security Rules* para proteger la integridad de la base de datos.
- **Core del Negocio (Gestión de Microaprendizaje):** Es el componente principal de la aplicación. Incluye la arquitectura para el streaming de videos cortos desde Youtube, el motor de cuestionarios (quizzes) para la validación de conocimientos y la lógica de sincronización offline/online del avance curricular.
- **Módulo de Reportes:** Generación de un tablero de métricas dentro de la aplicación móvil donde el docente podrá visualizar su progreso cualitativo y cuantitativo, horas de capacitación acumuladas y módulos completados con éxito.

- **Integraciones:** Sincronización de la aplicación con servicios de terceros para la reproducción multimedia y servicios de notificaciones *push* (Firebase Cloud Messaging) para alertar sobre nuevos contenidos o recordatorios de estudio.
- **Panel Administrativo:** Configuración y uso de la Consola de Firebase como entorno de gestión de contenidos y usuarios. A través de este panel, se realizará la carga de metadatos de los cursos, gestión de archivos multimedia y monitoreo de la salud del sistema.

5.2.- Limitaciones

- **Tiempo:** El ciclo de vida del desarrollo, desde el análisis hasta la evaluación de usabilidad con docentes, debe completarse en un periodo estricto de 4 meses, lo que prioriza el desarrollo de las funcionalidades críticas (MVP - Producto Mínimo Viable).
- **Infraestructura:** La solución operará bajo la capa gratuita (Spark Plan) de Firebase. Esto limita el número de conexiones simultáneas, la capacidad de almacenamiento multimedia y el ancho de banda mensual, por lo que el sistema no está diseñado inicialmente para un despliegue masivo a nivel nacional.
- **Recursos Humanos:** El diseño, desarrollo frontend, configuración de backend y redacción técnica son responsabilidad de un único desarrollador, lo que restringe la posibilidad de implementar versiones paralelas para otras plataformas (como web o desktop) en esta fase.
- **Accesos Institucionales:** El proyecto se desarrolla de manera independiente. Por tanto, no cuenta con integración directa a las bases de datos oficiales del Ministerio de Educación de Bolivia (como el sistema SIE), operando con una base de datos de usuarios autónoma y validada solo por el grupo de control de docentes participantes.

6.- Marco Tecnológico y Conceptual

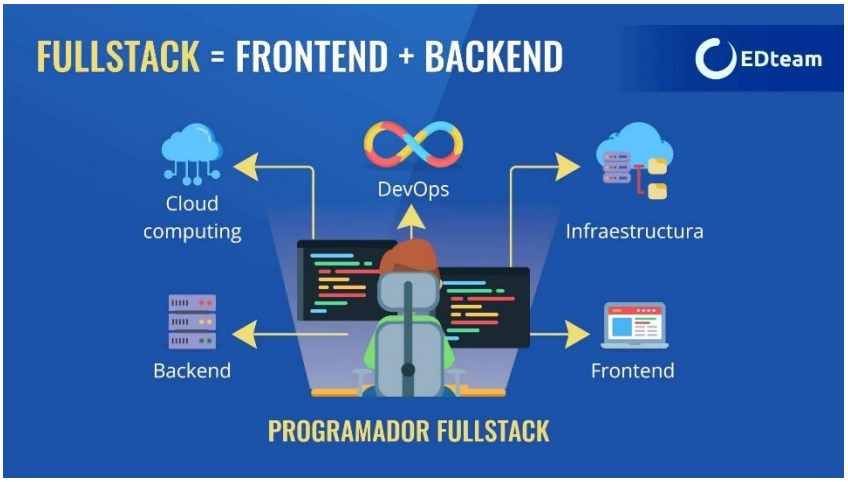
Desarrollo de aplicación móvil Full Stack

El desarrollo de aplicaciones móviles Full Stack constituye una metodología integral de ingeniería de software que aborda la creación completa de una aplicación, abarcando tanto el lado del cliente (frontend) como el lado del servidor (backend), incluyendo la lógica de negocio, las bases de datos y las API de conexión (Liao-Mäkinen, 2023). En el contexto móvil, este enfoque adquiere particular relevancia debido a la necesidad de integrar interfaces de usuario ejecutándose en dispositivos con sistemas backend alojados en la nube o servidores remotos.

Entre los beneficios destacan la eficiencia en los ciclos de desarrollo, la consistencia en el rendimiento entre plataformas y la optimización de los flujos de trabajo al unificar conocimientos y herramientas (Herath, 2024). Sin embargo, también se señalan desafíos críticos como la posible aparición de cuellos de botella en el rendimiento debido a las capas de abstracción de los frameworks multiplataforma, así como problemas de escalabilidad que requieren un diseño arquitectónico cuidadoso a medida que la aplicación crece en usuarios y funcionalidades (Ušaj, 2016). La correcta selección y combinación de tecnologías frontend, backend y bases de datos constituye, por tanto, un proceso de investigación fundamental para equilibrar la velocidad de desarrollo con el rendimiento y la escalabilidad del producto final.

Figura 1

Imagen Arquitectura Full Stack



Nota, Microaprendizaje (Microlearning): Qué Es y Por Qué Es Tan Efectivo en 2025. https://www.

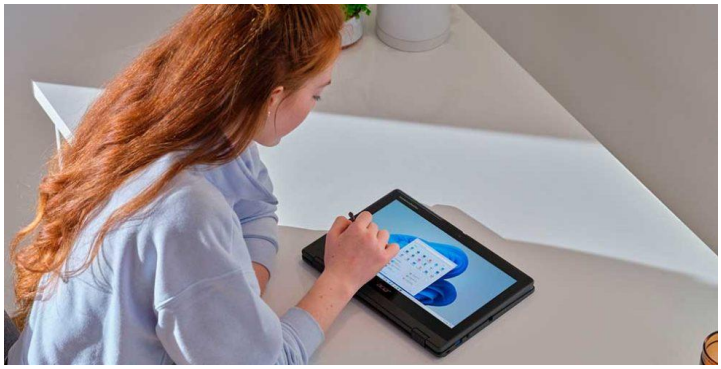
Cursos de microaprendizaje

El microaprendizaje es un enfoque educativo que se caracteriza por brindar información en porciones pequeñas, altamente enfocadas y de corta duración, idealmente entre 3 y 15 minutos. Esta metodología permite un consumo ágil de contenidos, adaptándose al ritmo del estudiante y facilitando el acceso a la

información en el momento preciso o cuando es más receptivo. Sus características principales incluyen la brevedad, la granularidad (cada unidad se centra en un solo tema), la variedad de formatos (videos, juegos, infografías) y la autonomía, permitiendo al alumno tomar el curso en cualquier momento y lugar. El microaprendizaje, a menudo informal y en línea, es efectivo para la renovación constante de conocimientos y la retención del aprendizaje, adaptándose a diversos estilos de aprendizaje (Durán Alcalá, M., & Escudero Nahón, A.,2023)

Figura 2

Imagen Microformaciones y aprendizaje personalizado



Nota, Una categoría de apps de aprendizaje personalizado.

https://www.muycomputerpro.com/2023/02/14/microsoft-learning-accelerators-apps-aprendizaje

Docente de educación secundaria en Bolivia

El rol del docente de educación secundaria en Bolivia, enmarcado en la Ley de Educación 070 "Avelino Siñani – Elizardo Pérez" y el Modelo Educativo Sociocomunitario Productivo, enfrenta desafíos significativos en el contexto actual. Investigaciones recientes evidencian una brecha educativa sustancial entre la formación secundaria y las exigencias de la educación superior, lo que responsabiliza al personal docente de formar bachilleres íntegros y capaces de enfrentar los retos universitarios (Quispe Condori, 2024). Esta situación se ve agravada por limitaciones en la formación docente para el uso efectivo de tecnologías de información y comunicación, donde los programas de capacitación no logran generar procesos de aprendizaje activo, sino que tienden a fomentar clases magistrales debido a problemas de equipamiento, conectividad y actualización pedagógica (Farfán, 2015, como se cita en Quispe Condori, 2024).

6.1.- Arquitectura de Software

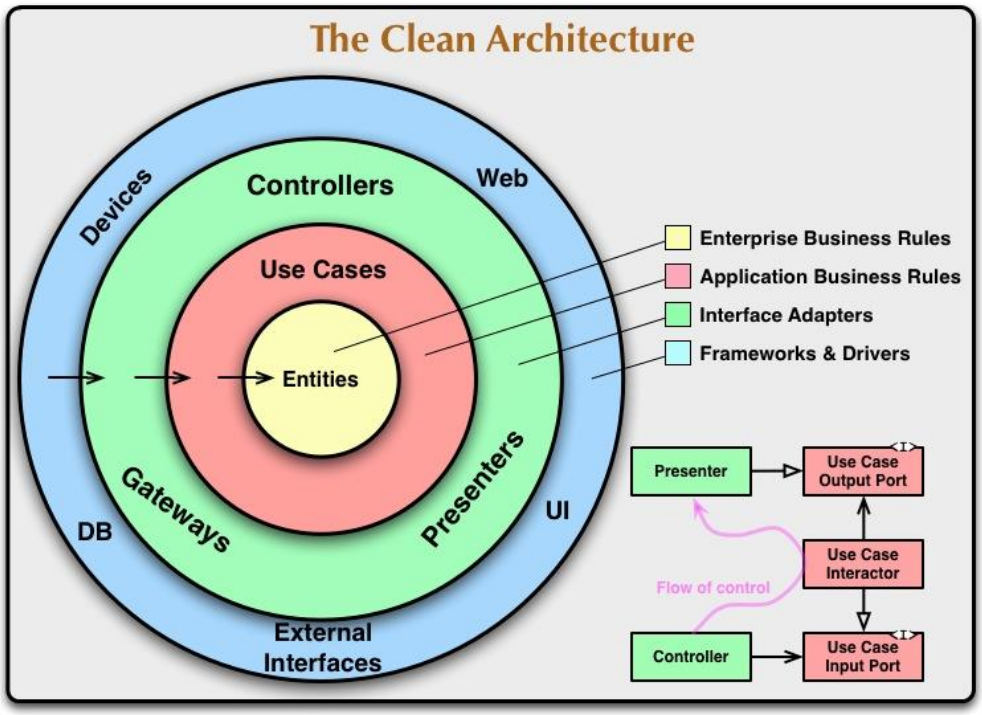
- Patrón arquitectónico (Clean Architecture)

Para asegurar la escalabilidad, mantenibilidad y robustez de estas aplicaciones, es fundamental adoptar principios de diseño arquitectónico sólidos. La Clean Architecture (Arquitectura Limpia), propuesta por Robert C. Martin, emerge como

una metodología clave para lograr estos objetivos. Esta arquitectura promueve una clara separación de las preocupaciones, organizando el código en capas concéntricas donde las dependencias solo fluyen hacia el interior. La aplicación se organiza en: Capa de Entidades (reglas de negocio), Casos de Uso, Adaptadores de Interfaz y Frameworks/Drivers. (Martin, 2017).

Figura 3

Imagen Clean Architecture



Nota, SOLID Principles and The Clean Architecture. https://kessler.tech/software-architecture/solid/

- Justificación técnica

Al aplicar Clean Architecture, las reglas centrales permanecen independientes de detalles externos como la base de datos (Firebase), la interfaz de usuario (Flutter) o cualquier framework específico. Esto se logra mediante la definición de interfaces claras (puertos) y su implementación (adaptadores) en las capas exteriores. La implementación de Clean Architecture junto con Flutter y Firebase facilita el desarrollo de aplicaciones móviles que no solo son potentes y rápidas, sino también flexibles y fáciles de mantener a largo plazo, permitiendo una adaptación ágil a futuros cambios y requerimientos.

6.2.- Stack Tecnológico

- Frontend (Flutter)

Flutter es un kit de desarrollo de software (SDK) de interfaz de usuario (UI) de código abierto, diseñado por Google, que permite a los desarrolladores crear aplicaciones compiladas de forma nativa para múltiples plataformas, incluyendo dispositivos móviles (Android e iOS), entornos web y sistemas de escritorio

(Windows, macOS y Linux), utilizando el lenguaje de programación Dart. Se distingue por su capacidad para ofrecer un rendimiento nativo gracias a la compilación directa a código ARM, y por su enfoque en widgets personalizables, donde cada elemento de la interfaz es un widget. Además, sus funcionalidades de Hot Reload y Hot Restart revolucionan el ciclo de desarrollo al permitir la visualización instantánea de cambios sin perder el estado de la aplicación, consolidándose como una opción eficiente y potente para el desarrollo multiplataforma (Google Flutter Documentation, s.f.)

Figura 4

Imagen Flutter

Nota, Flutter for startups: Build Mobile Apps with Flutter. https://www.solutelabs.com/blog/flutter-for-your-next-product-idea-everything-you-need-to-know

- Backend (Firebase)

Firebase es una plataforma Backend-as-a-Service (BaaS), desarrollada por Google, cuyo objetivo primordial es simplificar y acelerar el desarrollo de aplicaciones móviles y web al ofrecer un completo ecosistema de herramientas y servicios que eliminan la necesidad de gestionar una infraestructura de servidor compleja. Esta plataforma provee soluciones esenciales como Cloud Firestore y Realtime Database para bases de datos NoSQL con sincronización de datos en tiempo real, un sistema de autenticación robusto, servicios de hosting para contenido estático y dinámico, y Cloud Functions para la ejecución de lógica backend sin servidor. Adicionalmente, incluye tiene herramientas para almacenamiento de archivos y resolución de problemas de estabilidad lo que facilita un desarrollo ágil y escalable (Google Firebase Documentation, s.f.).

Figura 5

Imagen Firebase

Nota, Representación de la marca Firebase. https://firebase.google.com/brand-guidelines?hl=es-419

- Base de datos (Cloud Firestore)

Cloud Firestore se define como una base de datos flexible y escalable para el desarrollo en la nube y dispositivos móviles. A diferencia de las bases de datos relacionales tradicionales, Firestore utiliza un modelo de orientación a documentos (colecciones y documentos), lo que permite jerarquizar la información de los cursos de microaprendizaje de manera intuitiva.

Su arquitectura está diseñada para la sincronización en tiempo real, utilizando *web sockets* para empujar actualizaciones a los dispositivos de los docentes sin necesidad de peticiones HTTP constantes. Además, su motor de persistencia local es clave para el contexto boliviano: permite que el docente consuma lecciones en áreas con conectividad intermitente, sincronizando el progreso automáticamente una vez se restablece el acceso a internet. Según Kleppmann (2017), este tipo de sistemas distribuidos resuelven la complejidad de la consistencia eventual mediante protocolos de replicación eficientes.

- Autenticación (Firebase Authentication)

La gestión de identidades es un componente crítico que debe delegarse a servicios especializados para mitigar riesgos de seguridad. Firebase Authentication proporciona un ecosistema robusto que implementa protocolos estándar como OAuth 2.0 y OpenID Connect. Esto permite que la aplicación no solo gestione correos y contraseñas de forma segura (mediante hashing de contraseñas en servidores de Google), sino que también facilite la integración futura con sistemas de identidad institucional (como Google Workspace for Education).

Según Kanani (2022), la delegación de la autenticación a un proveedor de identidad (IdP) externo reduce la superficie de ataque del sistema, ya que el desarrollador nunca manipula directamente credenciales sensibles. Además, el servicio incluye mecanismos de detección de anomalías y protección contra ataques de fuerza bruta de manera nativa.

- Infraestructura y despliegue

El despliegue del proyecto adopta un modelo de Infraestructura como Servicio (IaaS) y Backend as a Service (BaaS) mediante Google Cloud Platform (GCP). Para la entrega del software, se utiliza Firebase App Distribution, lo que permite un ciclo de integración y entrega continua (CI/CD) simplificado. Esta herramienta facilita que las versiones de prueba lleguen a los docentes de secundaria de forma controlada antes del lanzamiento oficial en tiendas.

El alojamiento de activos (imágenes, videos cortos de microaprendizaje y scripts) se gestiona a través de Youtube, el cual ofrece redundancia geográfica y baja latencia. Como señala Forsgren et al. (2018) en su estudio sobre entrega de software de alto rendimiento, la automatización del despliegue y el uso de plataformas en la nube son predictores directos del éxito técnico y la estabilidad de una aplicación en entornos de producción.

6.3.- Seguridad

La seguridad en aplicaciones móviles de carácter educativo no solo protege la propiedad intelectual de los contenidos, sino que salvaguarda la integridad de la información personal de los docentes.

- Autenticación

La autenticación es el proceso de verificar la identidad de un usuario. En este proyecto, se implementa mediante tokens de acceso de corta duración bajo el estándar JSON Web Tokens (JWT) gestionados por Firebase. Este método asegura que la identidad del docente se valide en cada petición al backend sin exponer sus credenciales. Según Zaman e Isaac (2020**)**, el uso de proveedores de identidad consolidados permite mitigar ataques de robo de sesión y asegura que solo los usuarios registrados en el sistema puedan interactuar con la plataforma de microaprendizaje.

- Autorización

A diferencia de la autenticación, la autorización define qué acciones puede realizar un usuario ya identificado. Se implementa mediante el modelo de Control de Acceso Basado en Atributos (ABAC) a través de las Security Rules de Firebase. Estas reglas actúan como un firewall a nivel de base de datos, evaluando en tiempo real si el uid del solicitante tiene permisos de lectura o escritura sobre un documento específico. Según señala Kanani (2022), la validación en el lado del servidor es obligatoria, ya que las restricciones impuestas en el código del cliente (Flutter) pueden ser vulneradas por usuarios avanzados.

- Protección de datos

La protección de datos abarca tanto la información en tránsito como en reposo. Se utiliza TLS 1.2+ para cifrar la comunicación entre la app móvil y la nube, evitando ataques de Man-in-the-Middle (MitM). Para los datos en reposo, se hace uso de la encriptación AES-256 proporcionada por la infraestructura de Google Cloud. Además, en el dispositivo local, se implementa el almacenamiento seguro mediante el uso de KeyChain (iOS) y Keystore (Android), asegurando que los tokens de sesión no sean extraíbles. Kleppmann (2017) enfatiza que la integridad de los datos es un pilar de la confiabilidad del sistema, especialmente en arquitecturas que permiten el acceso offline.

- Buenas prácticas OWASP

El proyecto se alinea con el OWASP Mobile Application Security Verification Standard (MASVS). Se prioriza la mitigación de los riesgos del Mobile Top 10, tales como el uso inadecuado de la plataforma y el almacenamiento inseguro de datos. Según Meucci y Muller (2023), seguir las guías de OWASP garantiza que la aplicación sea resiliente ante intentos de ingeniería inversa y manipulación de archivos. Se implementa específicamente la validación de entradas de datos

para prevenir inyecciones y la configuración estricta de encabezados de seguridad en las funciones de backend.

7. Análisis del Sistema

El análisis del sistema permite definir las fronteras del software y las interacciones necesarias para cumplir con los objetivos de microaprendizaje docente. Este capítulo detalla desde quiénes interactúan con la plataforma hasta la lógica de los requerimientos funcionales expresados en historias de usuario.

7.1.- Actores del sistema

En un sistema Full Stack distribuido, los actores no son solo personas, sino cualquier entidad que interactúe con el sistema. Para este proyecto, se han identificado tres actores principales que operan bajo un modelo de Control de Acceso Basado en Roles (RBAC), garantizando la seguridad y la integridad de los datos en Firebase.

Actor	Tipo	Descripción y Responsabilidades
Docente (Usuario Final)	Humano / Externo	Maestro de educación secundaria en Bolivia. Es el consumidor principal de las microformaciones. Sus responsabilidades incluyen: gestionar su perfil, visualizar contenidos multimedia, realizar evaluaciones rápidas y realizar el seguimiento de su propio progreso de aprendizaje.
Administrador de Contenidos	Humano / Interno	Personal técnico o pedagógico encargado de la gestión de la plataforma. Sus responsabilidades incluyen: el alta de nuevos módulos de microaprendizaje, la gestión de usuarios, el monitoreo de métricas de participación y la actualización de materiales en Cloud Firestore.
Servicio de Firebase (Sistema)	No Humano / Sistema	Actor automático que actúa como proveedor de servicios de backend. Es responsable de la persistencia de datos, la sincronización en tiempo real, el envío de notificaciones push de recordatorio y la ejecución de reglas de seguridad para validar las peticiones del Frontend.

7.2 Requerimientos

7.2.1. Requerimientos funcionales

- **RF1**: Registro y autenticación de usuarios (maestros y administradores) con correo y contraseña.
- **RF2**: Búsqueda de microformaciones por título o categoría mediante consultas de texto.

- **RF3**: Visualización de microformaciones con contenido multimedia (texto, videos, cuestionarios).
- **RF4**: Seguimiento del progreso del usuario, mostrando porcentaje completado por curso.
- **RF5**: Generación de reportes en PDF con el resumen de microformaciones completadas.
- **RF6**: Gestión de microformaciones (crear, editar, eliminar) por parte de administradores.

7.2.2. Requerimientos no funcionales

- **Usabilidad**: Interfaz intuitiva, diseñada para usuarios con poca experiencia tecnológica.
- **Rendimiento**: El tiempo de carga será el menor posible dependiendo la velocidad de la conexión de internet.
- **Soporte offline**: Acceso al sistema sin conexión, con sincronización al reconectar.
- **Seguridad**: Autenticación segura y protección de datos en Firebase.
- **Compatibilidad**: Funcionamiento en Android.

7.3.- Historias de Usuario

Las historias de usuario representan las necesidades de los actores del sistema. Cada una incluye un **Criterio de Aceptación** técnico, asegurando que se cumplan los estándares de arquitectura limpia y seguridad propuestos.

HU-01: Registro y Autenticación de Seguridad

- **Como:** Docente de secundaria.
- **Quiero:** Acceder al sistema mediante usuario y contraseña.
- **Para:** Tener un perfil personalizado donde se guarde mi progreso de capacitación de forma segura.
- **Criterios de Aceptación:**
    1. El sistema debe validar las credenciales mediante **Firebase Authentication**.
    2. Al iniciar sesión, la app debe recibir un **JWT (JSON Web Token)** para autorizar peticiones posteriores.
    3. Debe permitir la recuperación de contraseña de forma automatizada.

HU-02: Visualización de Catálogo de Microformaciones

- **Como:** Docente de secundaria.
- **Quiero:** Ver un listado organizado de los cursos cortos disponibles.
- **Para:** Seleccionar el tema que más se adapte a mi necesidad pedagógica inmediata.
- **Criterios de Aceptación:**
    1. La lista de cursos debe cargarse desde **Cloud Firestore**.
    2. La interfaz en **Flutter** debe permitir el scroll fluido y mostrar una miniatura (thumbnail) del curso.
    3. El sistema debe permitir filtrar por áreas (Matemáticas, Física, Lenguaje, etc.).

HU-03: Consumo de Microaprendizaje (Video/Contenido)

- **Como:** Docente de secundaria.
- **Quiero:** Reproducir videos cortos (5-15 min) y leer material complementario.
- **Para:** Actualizar mis conocimientos en momentos libres sin saturar mi agenda.
- **Criterios de Aceptación:**
    1. Los videos deben reproducirse mediante un reproductor integrado en Flutter que soporte streaming desde Youtube.
    2. El sistema debe registrar automáticamente el porcentaje de avance en la base de datos.
    3. Debe funcionar correctamente incluso si la conexión a internet es inestable (caché local).

HU-04: Evaluación de Conocimientos (Quiz)

- **Como:** Docente de secundaria.
- **Quiero:** Responder un cuestionario breve al finalizar cada módulo.
- **Para:** Validar lo aprendido y obtener una retroalimentación inmediata.
- **Criterios de Aceptación:**
    1. Las preguntas deben cargarse dinámicamente desde una colección de Firestore.
    2. Al finalizar, el sistema debe calcular la nota en el backend y mostrar el resultado en tiempo real.
    3. Si se aprueba, el estado del módulo debe cambiar a "Completado".

HU-05: Seguimiento de Progreso y Logros

- **Como:** Docente de secundaria.
- **Quiero:** Visualizar una barra de progreso o un panel de estadísticas de mis cursos.
- **Para:** Mantener la motivación y llevar un control de mi formación continua.
- **Criterios de Aceptación:**

1. El panel de usuario debe mostrar datos agregados (cursos iniciados, terminados y horas acumuladas).
2. La actualización del progreso debe ser reactiva (uso de StreamBuilder en Flutter).
3. Los datos deben persistir entre sesiones de usuario.

HU-07: Acceso a Material de Lectura y Estrategias Pedagógicas

- **Como:** Docente de secundaria.
- **Quiero:** Acceder a guías breves y documentos de estrategias didácticas en formato digital.
- **Para:** Consultar técnicas específicas que pueda aplicar inmediatamente en el aula sin necesidad de revisar todo el material audiovisual.
- **Criterios de Aceptación:**
    1. La aplicación debe permitir la visualización de documentos (PDF o Markdown) que se generarán con información almacenada.
    2. El material debe estar categorizado por "Estrategias de Aula", "Evaluación" o "Herramientas Digitales".
    3. El sistema debe permitir la lectura sin conexión (Offline) una vez que el documento ha sido abierto por primera vez (Caché local).
    4. Debe existir un marcador de "Lectura completada" que se sincronice con el perfil del docente en **Firestore**.

HU-07: Gestión de Contenidos (Admin)

- **Como:** Administrador del sistema.
- **Quiero:** Subir nuevos videos y crear cuestionarios desde un panel de gestión.
- **Para:** Mantener la oferta formativa actualizada para los maestros.
- **Criterios de Aceptación:**
    1. Solo usuarios con el rol de "Admin" en sus Custom Claims de Firebase pueden acceder a estas funciones.
    2. El sistema debe permitir la carga de archivos multimedia a Youtube.
    3. Se debe validar que los campos obligatorios del curso estén completos antes de publicar.

7.4 Modelado del sistema

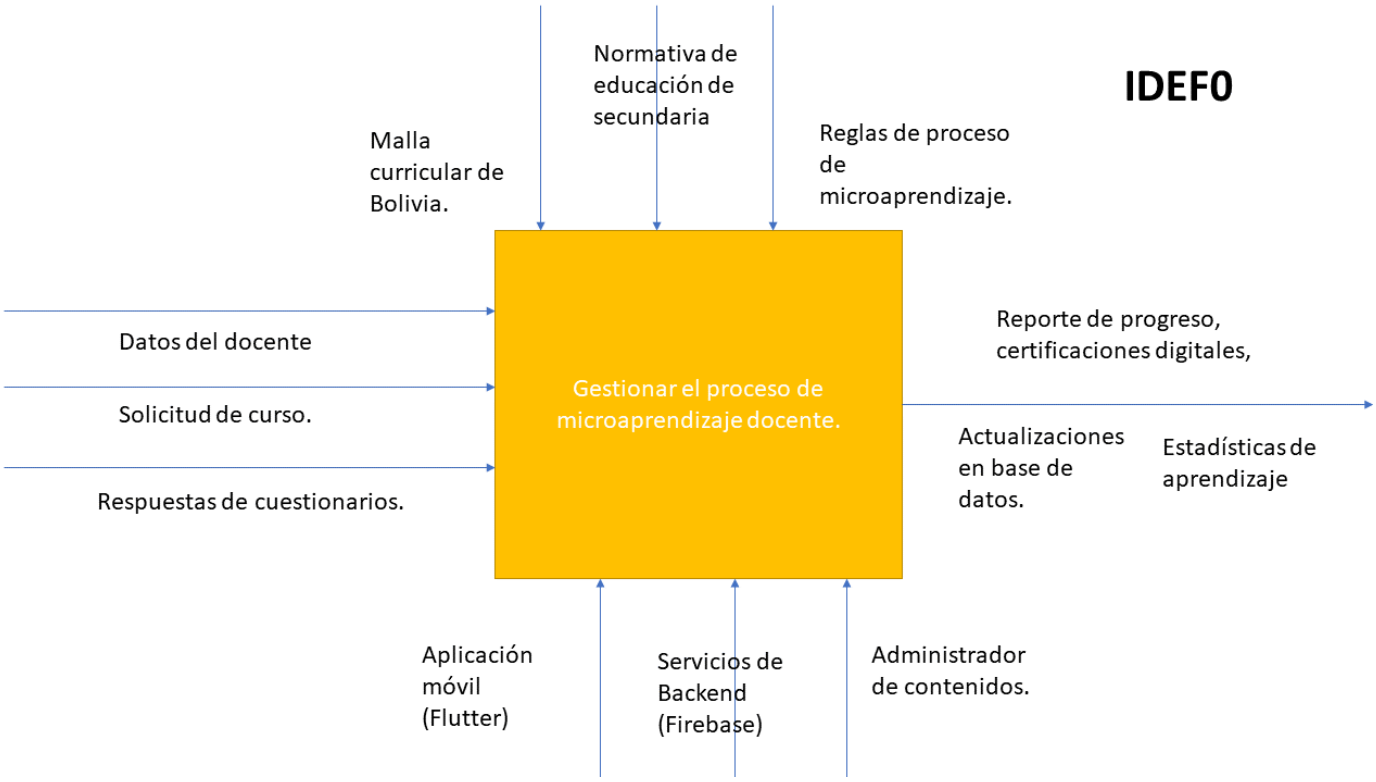
- IDEF0

El estándar IDEF0 se utiliza para modelar las decisiones, acciones y actividades de un sistema. Se centra en las funciones y utiliza entradas, controles, salidas y mecanismos (ICOM).

Diagrama de Nivel 0 (Caja Negra):

- **Función Principal (Centro):** Gestionar el proceso de microaprendizaje docente.

- **Entradas (Inputs):** Datos del docente, solicitud de curso, respuestas de cuestionarios.
- **Controles (Controls):** Normativa de educación de secundaria, malla curricular de Bolivia, reglas de negocio de microaprendizaje, límites de la capa gratuita de Firebase.
- **Salidas (Outputs):** Reporte de progreso, certificaciones digitales, estadísticas de aprendizaje, actualizaciones en base de datos.
- **Mecanismos (Mechanisms):** Aplicación móvil (Flutter), Servicios de Backend (Firebase), Administrador de contenidos.



Descripción del Proceso de Gestión (IDEF0)

El proceso central, "Gestionar el proceso de microaprendizaje docente", opera como una unidad de transformación sistémica detallada a continuación:

1. Entradas (Input)

Representan los elementos que el sistema procesa para generar un resultado:

- Datos del docente: Información de perfil y registro necesaria para personalizar la experiencia.
- Solicitud de curso: La acción explícita del usuario al elegir un contenido específico del catálogo.

- Respuestas de cuestionarios: Datos generados por el docente durante la evaluación, que sirven como materia prima para determinar el progreso.

2. Mecanismos (Mechanism)

Son los recursos tecnológicos y humanos necesarios para ejecutar la actividad:

- Aplicación móvil (Flutter): El medio de interacción frontal que permite al docente visualizar y responder a los contenidos.
- Servicios de Backend (Firebase): La infraestructura que soporta el almacenamiento, la lógica de negocio y la seguridad.
- Administrador de contenidos: El actor encargado de curar y subir los módulos pedagógicos al sistema.

3. Controles (Control)

Son las guías, leyes y reglas que restringen y dirigen el proceso para que sea válido:

- Malla curricular de Bolivia: Asegura que los contenidos sean pertinentes al contexto educativo nacional.
- Normativa de educación secundaria: Marco legal que rige la formación docente.
- Reglas de proceso de microaprendizaje: Parámetros lógicos (tiempos, puntajes mínimos, secuencias) que definen cómo se debe avanzar en la plataforma.

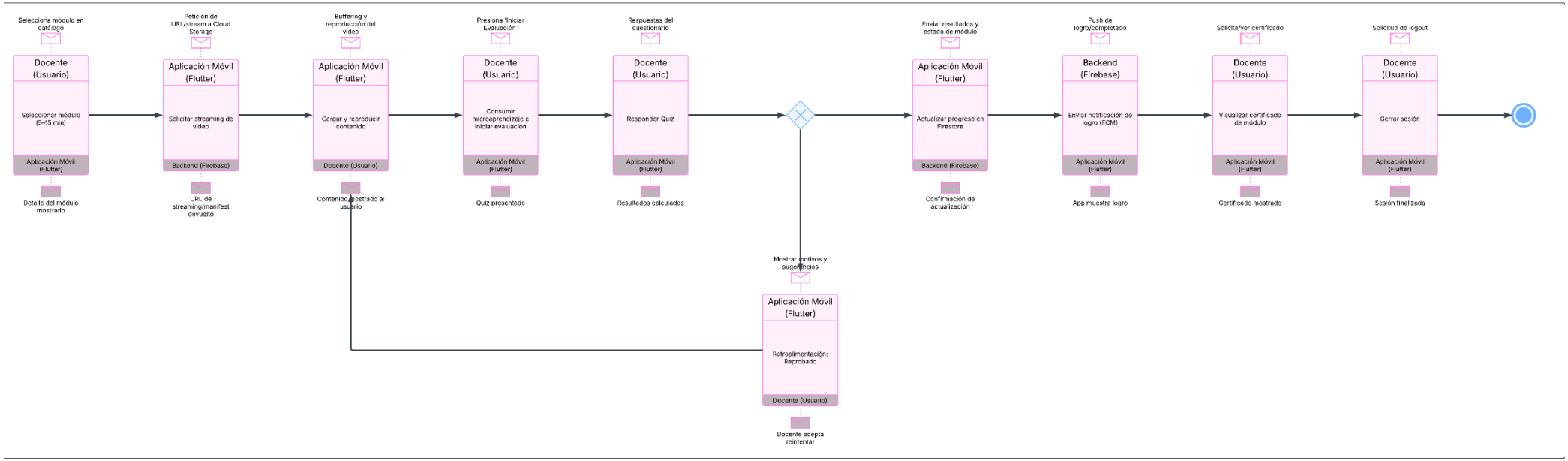
4. Salidas (Output)

Es el valor agregado resultante tras la ejecución del proceso:

- Reporte de progreso y certificaciones digitales: Documentos que avalan la competencia adquirida por el docente.
- Actualizaciones en base de datos: Sincronización del estado de avance en los servidores.
- Estadísticas de aprendizaje: Datos analíticos que permiten medir el impacto y la efectividad de los contenidos.

- BPMN

A diferencia del IDEF0, el BPMN muestra la secuencia lógica y los tiempos del proceso. El proceso clave es el "Ciclo de Capacitación por Microaprendizaje".



Explicación del Proceso de Microaprendizaje

El flujo operativo del sistema se divide en cuatro etapas principales que garantizan el ciclo de formación del docente:

a. Consumo de Contenido Multimedia

El proceso inicia cuando el docente selecciona un módulo del catálogo en la aplicación. En este punto, la aplicación Flutter realiza una petición de URL de streaming a Firebase Cloud Storage. Una vez devuelto el manifiesto o enlace, la aplicación gestiona el buffering y la reproducción del video educativo, permitiendo que el docente consuma el contenido pedagógico de forma asíncrona.

b. Evaluación y Toma de decisiones

Finalizado el video, el docente inicia la evaluación del módulo. El sistema presenta un cuestionario dinámico (Quiz) cuyas respuestas son procesadas por la lógica de negocio en Flutter. El flujo llega a una compuerta de decisión (Gateway) basada en los resultados:

- Si es reprobado: El sistema muestra retroalimentación y sugerencias, redirigiendo al docente a visualizar nuevamente el contenido para reforzar el aprendizaje.
- Si es aprobado: El flujo continúa hacia la fase de registro de logros.

c. Persistencia de Datos y Notificaciones

Tras la aprobación, la aplicación envía los resultados y el estado del módulo al backend. Firebase Firestore actualiza el progreso del usuario de forma atómica. Simultáneamente, se activa un servicio de Cloud Messaging (FCM) que envía una notificación push de logro o módulo completado, reforzando la gratificación inmediata del usuario por haber concluido el microaprendizaje.

d. Certificación y Cierre

En la etapa final, el docente solicita visualizar su certificado de módulo. La aplicación Flutter recupera el documento generado y lo muestra en pantalla. Finalmente, el proceso concluye cuando el docente decide cerrar su sesión, lo cual gatilla la limpieza de estados en la aplicación y finaliza el ciclo operativo de manera segura

8.- Diseño del Sistema

El diseño del sistema establece la estructura técnica que permite la operatividad de la plataforma. Se utiliza una notación estandarizada para describir la interacción entre las interfaces móviles, los servicios de backend y la infraestructura en la nube.

8.1.- Arquitectura General

8.1.1. WAE UML Extendido

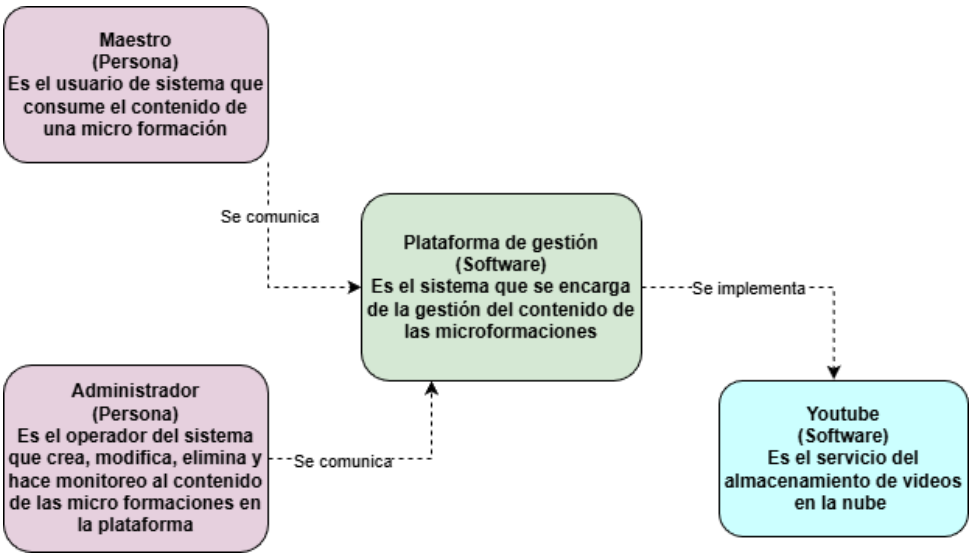
Aunque el proyecto es una aplicación móvil nativa con Flutter, se utiliza el modelo WAE para representar la naturaleza del sistema como una aplicación distribuida que consume recursos web. Este diagrama muestra la separación entre los componentes lógicos del cliente y los recursos del servidor (Firebase).

- Lado Cliente (Mobile App): Representado por las pantallas de Flutter, el gestor de estado (Provider/Bloc) y la base de datos local (Cache).
- Lado Servidor (Cloud): Representado por los endpoints de Firebase Auth, Firestore y Storage.
- Relación: Se modela mediante asociaciones "Client-Server" utilizando protocolos HTTPS/WSS.

8.1.2. Diagrama C4 (Contexto, Contenedores, Componentes)

Nivel 1

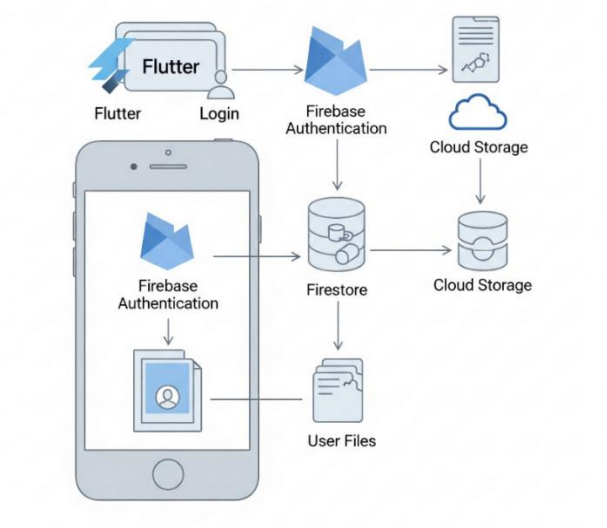
Muestra el sistema de Microaprendizaje interactuando con el Docente y con los sistemas externos (Youtube).



Nivel 2

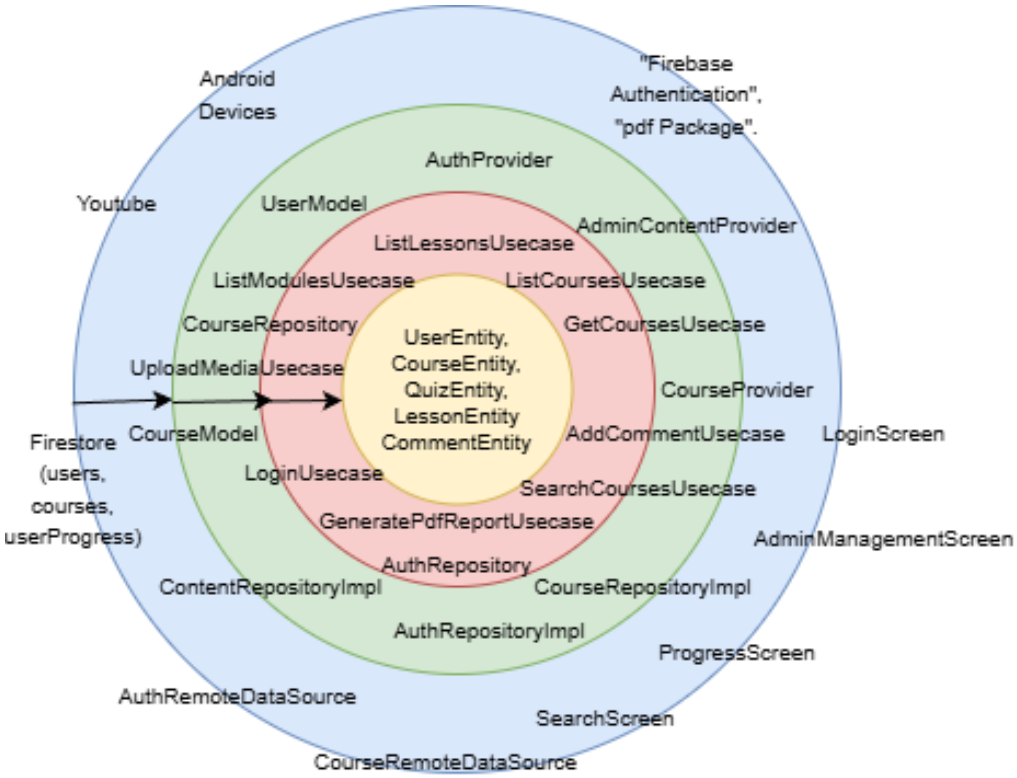
Contenedores: Describe cómo se divide la solución.

- **Mobile App (Flutter):** El contenedor que el docente descarga.
- **Database (Firestore):** Contenedor de datos NoSQL.
- **File Storage (Youtube):** Contenedor de recursos multimedia.
- **Auth Service (Firebase Auth):** Contenedor de gestión de identidad.



Nivel 3

Utilizando Clean Architecture tenemos el siguiente diagrama:



Fuente, Elaboración Propia

- **Cliente (Flutter)**: Pantallas para login, búsqueda, visualización de cursos, progreso y reportes.
- **Backend (Firebase)**: Authentication verifica usuarios, Firestore almacena datos, YouTube provee multimedia.
- **Conexión**: APIs de Firebase para comunicación cliente-servidor.

Análisis por capas del Clean Architecture

Las dependencias solo apuntan hacia adentro. La capa 4 depende de la 3, la 3 de la 2, y la 2 de la 1. La capa 1 no depende de nadie.

Capa 1: Enterprise Business Rules (Reglas de Negocio de la Empresa)

Esta es la capa más interna y pura. Contiene los objetos de negocio más críticos y las reglas que son universales para la empresa, independientemente de la aplicación que se esté construyendo.

- **Color: Amarillo**
- **Componentes del proyecto:**
  - **Entities:**

    UserEntity, CourseEntity, LessonProgressEntity, etc. Son el corazón del dominio, representando los conceptos fundamentales sin depender de nada.

Capa 2: Application Business Rules (Reglas de Negocio de la Aplicación)

Esta capa orquesta el flujo de datos y contiene las reglas específicas de la aplicación. Define cómo se manipulan las entidades para cumplir con los casos de uso.

- **Color: Rojo**
- **Componentes del proyecto:**
  - **Use Cases:**

    LoginUsecase, GetCoursesUsecase, GenerateAllUsersReportUse case, etc. Cada uno encapsula una interacción específica del usuario o del sistema.

  - **Repository Interfaces (Contratos):**

    AuthRepository, CourseRepository, ContentRepository. Son las abstracciones que los casos de uso utilizan para acceder a los datos, definiendo *qué* datos se necesitan, pero no *cómo* se obtienen.

Capa 3: Interface Adapters (Adaptadores de Interfaz)

Esta capa actúa como un conjunto de convertidores. Adapta los datos del formato más conveniente para los frameworks (capa 4) al formato más conveniente para las reglas de negocio (capas 1 y 2), y viceversa.

- **Color: Verde**
- **Componentes de tu proyecto:**
  - **Providers:**

    AuthProvider, CourseProvider, AdminContentProvider, etc. Actúan como *Presenters* o *Controllers*. Toman la entrada de la UI, la pasan a los *Use Cases* y luego toman la salida para formatearla como estado para la UI.
  - **Repository Implementations:**

    AuthRepositoryImpl, CourseRepositoryImpl. Son la implementación concreta de los contratos definidos en la capa 2. Adaptan las llamadas del repositorio a una fuente de datos específica.
  - **Models:**

    UserModel. Estas clases son parte del proceso de adaptación. Convierten los datos crudos de Firebase (un Map<String, dynamic>) en objetos Dart estructurados que la implementación del repositorio puede usar antes de convertirlos en Entities.

Capa 4: Frameworks & Drivers (Frameworks y Controladores)

Esta es la capa más externa. Está compuesta por todo lo que es externo a la aplicación: la base de datos, el framework de UI, las bibliotecas de terceros, etc. Aquí es donde los detalles de implementación viven.

- **Color: Azul**
- **Componentes de tu proyecto:**
  - **UI (Framework):**

    LoginScreen, HomeScreen, BackgroundImage y todos los Widgets de Flutter. Es el framework que dibuja la pantalla.
  - **Drivers (Base de Datos/Servicios):**

    FirebaseAuth, FirebaseFirestore, FirebaseStorage. Son los controladores externos que realmente se comunican con los servidores de Google.
  - **Data**

    Sources: AuthRemoteDataSource, CourseRemoteDataSource.

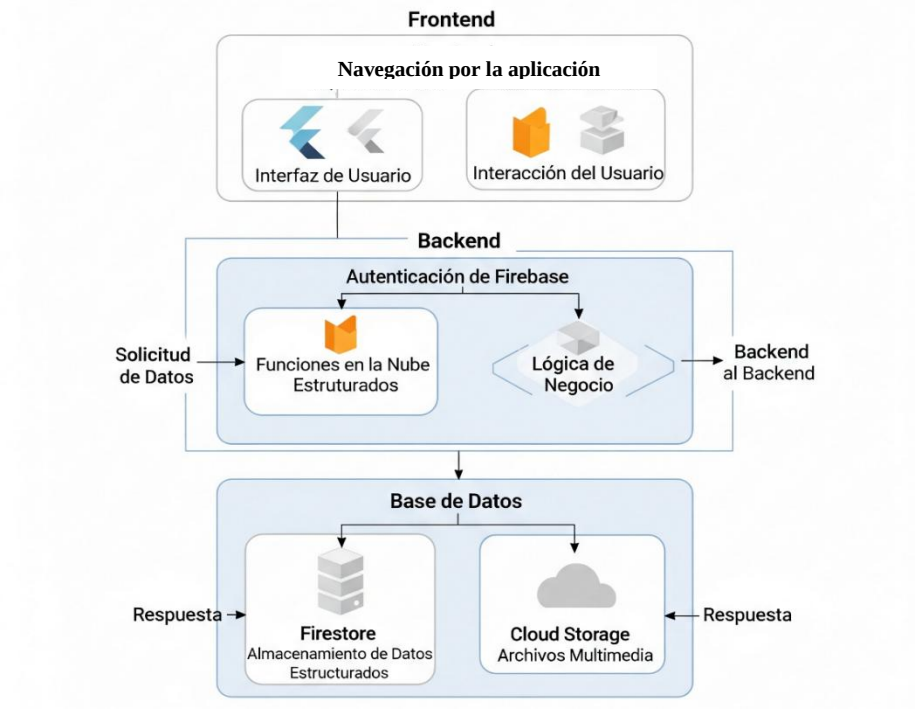
    Son las clases que directamente usan los *Drivers* (Firebase) para obtener o enviar datos.
  - **Infrastructure (Framework):** El archivo main, la configuración de MaterialApp, el contenedor de inyección de dependencias y cualquier otra configuración a nivel de framework.

Este modelo de cuatro capas te da una visión aún más granular y estricta de la separación de responsabilidades, lo cual es fantástico para la mantenibilidad y la capacidad de prueba de tu aplicación.

9. Flujo entre frontend, backend y servicios

La arquitectura sigue un modelo cliente-servidor, con los siguientes componentes:

- **Cliente**: Aplicación móvil desarrollada en Flutter, que proporciona la interfaz de usuario para maestros y administradores. Incluye pantallas para autenticación, búsqueda, visualización de microformaciones, seguimiento de progreso y generación de reportes.
- **Servidor**: Firebase como backend, que incluye:
  - **Firebase Authentication**: Gestión de autenticación de usuarios.
  - **Firebase Firestore**: Base de datos NoSQL para almacenar datos de usuarios, microformaciones y progreso.
  - **Firebase Storage**: Almacenamiento de contenido multimedia (imágenes).
- **Servicios externos**:
  - Paquete **pdf** de Flutter para generar reportes en PDF.
  - Paquete **youtube_player_flutter** para reproducción de videos en la app, obteniendo el contenido desde Youtube.



El usuario (maestro o administrador) accede a la app móvil, se autentica mediante Firebase Authentication, interactúa con la interfaz Flutter para buscar o gestionar microformaciones, y los datos se almacenan o recuperan de Firestore. Los contenidos multimedia se obtendrán desde Youtube, con soporte offline para acceder a textos relacionados a los cursos.

8.2.- Modelo de Datos

A diferencia de los sistemas relacionales que utilizan tablas rígidas, este proyecto emplea una base de datos NoSQL donde la escalabilidad y la velocidad de lectura son prioritarias. El modelo se ha diseñado siguiendo principios de desnormalización controlada para optimizar el consumo de la capa gratuita de Firebase, minimizando el número de lecturas necesarias por cada pantalla.

- Diagrama ER o Modelo Relacional

Aunque Firestore no aplica integridad referencial de forma nativa (como llaves foráneas), se presenta el diagrama ER para representar la vinculación lógica entre los datos del docente y los contenidos formativos.

Descripción de las Colecciones Principales:

- **Users (Docentes):** Almacena el perfil del docente. Se utiliza el UID generado por Firebase Auth como llave primaria.
- **Courses (Cursos):** Contiene la información general de las áreas de capacitación (ej. "Técnicas de Evaluación").
- **Lessons (Lecciones/Módulos):** Es una sub-colección o colección vinculada que contiene los metadatos de los videos y el contenido de microaprendizaje.
- **Progress (Avance):** Una colección puente que registra la relación muchos-a-muchos entre Docentes y Lecciones, guardando el puntaje del quiz y el estado de finalización.

8.3.- Diseño de Interfaces

A continuación, se ve el diseño de los mockups realizado en el software Figma:

Login y Registro del sistema

Figura 15

Mockup 1

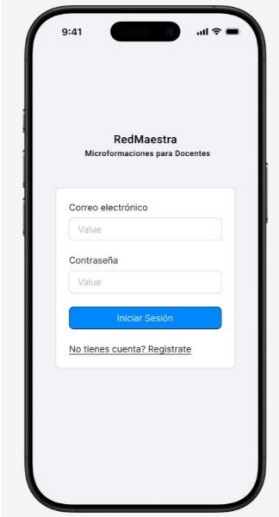
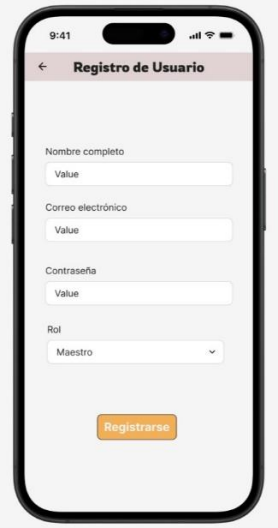


Figura 16

Mockup 2



Se muestra los Mockups de la página de inicio de la aplicación y registro de usuario, en donde se puede apreciar el diseño que se tendrá el prototipo.

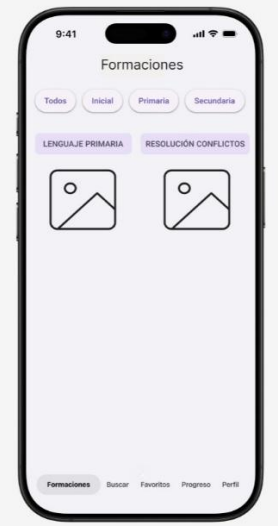
Páginas del sistema (Rol de maestro)

Figura 17

Mockup 3

Figura 18

Mockup 4



Se presenta los Mockups de la sección de Buscar y Formaciones

Figura 19

Mockup 5

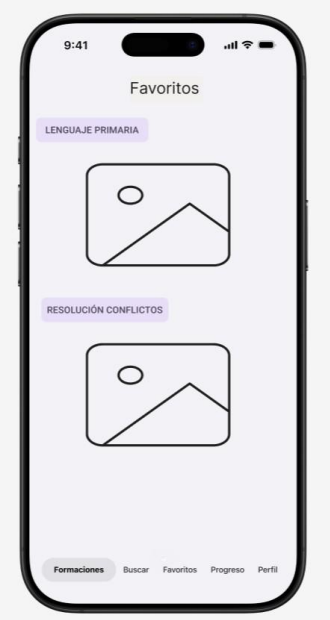


Figura 20

Mockup 6

Se puede apreciar los Mockups de la sección de Favoritos y Progreso

Figura 21

Mockup 7

Figura 22

Mockup 8

Se puede observar los Mockups de la sección de Perfil y la pantalla de confirmación para comenzar con un curso

Sección de lecciónes (Rol de maestro)

Figura 23

Mockup 9

Figura 24

Mockup 10

A continuación, mostramos los diseños preliminares (mockups) de la página donde se observan las diferentes lecciones disponibles y el contenido de cada una.

Figura 25

Mockup 11

Figura 26

Mockup 12

Se muestran los mockups del cuestionario y el reproductor de video de la lección vistos como rol de “Maestro”

Páginas del sistema (Rol de administrador)

Figura 27

Mockup 13

Figura 28

Mockup 14

Se observa los mockups la página principal después de ingresar como Administrador. También de la sección de gestión del contenido de la app.

Figura 29

Mockup 15

Figura 30

Mockup 16

Se puede apreciar los mockups del contenido de las lecciones distribuidos en módulos y cuando desplegamos la ventana para gestionar el video respectivo.

Figura 31

Mockup 17

Figura 32

Mockup 18

A continuación, mostramos los mockups cuando ingresamos a la opción de comentarios, luego cuando desplegamos la edición de datos del módulo.

Figura 33

Mockup 19

Figura 34

Mockup 20

Se muestra los mockups cuando desplegamos la ventana de modificación de información de la lección y la ventana de gestión de estrategias correspondientes.

8.4.- Maquetado

- Diagrama tecnológico

El sistema se fundamenta en una arquitectura híbrida que aprovecha la potencia del procesamiento local en dispositivos móviles y la escalabilidad de la computación en la nube. Esta interacción se detalla a continuación:

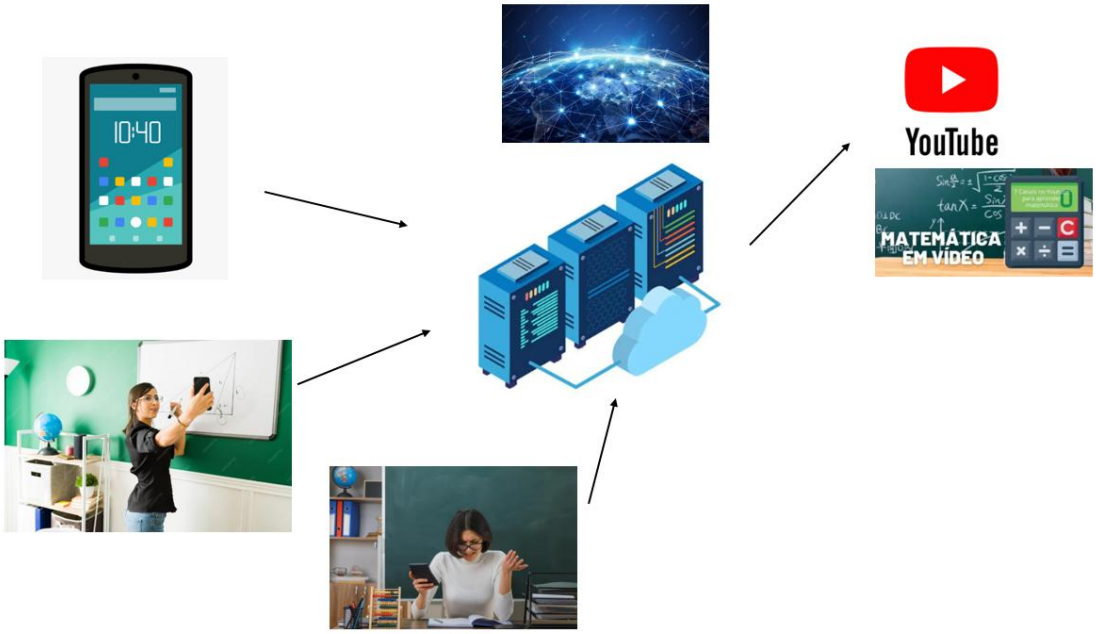
1. Dominio del Cliente: Interfaz e Interacción (Frontend)

En este nivel, la aplicación desarrollada en Flutter actúa como el punto de contacto principal para los maestros de secundaria en Bolivia. Al ser un framework multiplataforma, permite una ejecución eficiente en diversos modelos de hardware, gestionando la lógica de presentación y el estado local de la aplicación. El dispositivo móvil del docente no solo sirve como visor, sino que realiza peticiones asíncronas hacia el exterior a través de protocolos de red seguros (HTTPS).

2. Entorno de Nube: Backend as a Service (BaaS)

La infraestructura de Firebase, sustentada por Google Cloud Platform (GCP), elimina la necesidad de gestionar servidores físicos o virtuales.

- Gestión de Datos: Cuando el docente interactúa con la aplicación, Flutter se comunica con Cloud Firestore (base de datos NoSQL). Aquí se almacenan las estructuras de las lecciones, los registros de progreso y las mallas curriculares mencionadas en el modelo IDEF0.
- Sincronización: Firebase permite que cualquier actualización en el catálogo de microformaciones se refleje de forma casi instantánea en el celular del maestro, optimizando el consumo de datos móviles.



En el diagrama podemos observar como los maestros de secundaria mediante su celular acceden a una aplicación que se conecta a internet, específicamente a los servidores de google porque es donde está almacenado nuestra Base de datos en Firebase, con las diferentes lecciones de micro formaciones. A la vez también se puede observar que el contenido multimedia es extraído del servicio de Youtube porque la plataforma tiene un gran desempeño en esta área.

3. Integración Multimedia y Consumo de Contenido

Un componente crítico del ecosistema es la delegación del contenido pesado:

- YouTube como Repositorio: En lugar de saturar el almacenamiento del backend propio, el sistema utiliza a YouTube como el motor de entrega de video (Content Delivery Network).
- Rendimiento: Esta integración garantiza que los docentes puedan visualizar las lecciones de microaprendizaje incluso en condiciones de ancho de banda limitado, aprovechando el streaming adaptativo de YouTube. La aplicación Flutter embebe estos recursos de forma transparente, manteniendo al docente dentro del entorno educativo del proyecto.

4. Conectividad y Seguridad

Todo el flujo de información depende de una conexión a internet activa que vincula al usuario boliviano con los centros de datos de Google. Esta comunicación está protegida por capas de autenticación que aseguran que solo los docentes registrados puedan acceder a las certificaciones y reportes de progreso generados como salida del proceso.

Bibliografía

- Herath, I. (2024). Cross-platform development with full-stack frameworks: Bridging the gap for seamless integration [Tesis de maestría, Metropolia University of Applied Sciences]. Theseus.

- Liao-Mäkinen, S. (2023). Developing a full stack mobile application [Tesis de licenciatura, Metropolia University of Applied Sciences]. Theseus.

- Ušaj, E. (2016). Technology stacks and frameworks for full-stack application development [Tesis de ingeniería, University of Ljubljana]. Faculty of Computer and Information Science Repository.

- Quispe Condori, Y. (2024). Brecha educativa entre estudiantes de Secundaria y Universidad Pública de El Alto. Alternancia Revista de Educación e Investigación, 6(10), 86-103.

- Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.

- Chai, N. (2020). Beginning Firebase: Build Real-world Applications with Google's Mobile Platform. Apress.

- Kanani, P. (2022). Cloud Security with Firebase: Architecting Secure Mobile Applications. Packt Publishing.

- Hardt, D. (2012/2015 update). The OAuth 2.0 Authorization Framework. IETF RFC 6749.

- Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution Press.

- Buyya, R., et al. (2018). Cloud Computing: Principles and Paradigms. Wiley.

- **Zaman, S., & Isaac, J. (2020).** *Firebase Essentials: Android Edition*. CreateSpace Independent Publishing Platform.

- **Bertocci, V. (2019).** *Modern Authentication with Azure Active Directory for Web Applications*. Microsoft Press. (Referencia para protocolos JWT y OpenID).

- **Kanani, P. (2022).** *Cloud Security with Firebase: Architecting Secure Mobile Applications*. Packt Publishing.

- **Moraes, R. (2018).** *Expert Firebase for Android: Master the Skills to Build High-Quality Apps*. Packt Publishing.

- **Kleppmann, M. (2017).** *Designing Data-Intensive Applications*. O'Reilly Media.

- **Ozkaya, E. (2019).** *Cybersecurity: The Beginner's Guide*. Packt Publishing.

- Meucci, P., & Muller, C. (2023). OWASP Mobile Application Security Guide. OWASP Foundation.

- Deogirikar, J. (2018). Security in the World of Mobile Applications. International Journal of Scientific Research and Managemen